OPEN IT SUMMER

BOOTCAMP

DAY 2

C# Fundamentals & .NET Web APIs

Syntax • OOP • Collections • LINQ • ASP.NET Core • Routing • Postman

Duration: ~2 hrs | Console app → Working Web API → Postman testing

Bootcamp Roadmap — Where Day 2 Fits

Day 2: C# + Web API intro → Day 3: Middleware & layering → Day 4: DI & MVC → Day 5: Full CRUD → Day 7: EF Core + DB

Session Overview

01	**Why C# & .NET?** Typed, OOP, cross-platform, .NET ecosystem
02	**Variables, Types & Control Flow** int, string, bool, var, null, loops, switch
03	**Methods & Classes (OOP)** Properties, constructors, methods, access modifiers
04	**Collections & LINQ** List<T>, Dictionary, Where, Select, OrderBy, lambdas
05	**ASP.NET Core Setup** dotnet CLI, project structure, Program.cs explained
06	**Controllers & Attribute Routing** [HttpGet/Post/Put/Delete], [FromBody], [FromQuery]
07	**First Web API + Testing** Returning JSON, HTTP status codes, Postman testing
08	**Data Annotations & Validation** [Required], [StringLength], [Range] — auto 400 Bad Request
09	**2-Hour Hands-On Lab** Student Grades Console + Book Inventory API

Why C# & .NET?

Foundation for Days 2–12

A strongly-typed, compiled, cross-platform ecosystem

Strongly Typed

Declare the type of every variable. The compiler catches type errors before the program runs — not in production at 2am.

True OOP

Classes, interfaces, inheritance, generics built natively. Not "JavaScript classes" — real type-based OOP.

Cross-Platform

.NET 8 runs identically on Windows, Linux, macOS. Same binary. No rewrites. Deploy to any cloud.

ASP.NET Core

Microsoft's high-performance web framework. Stack Overflow, Bing, Azure use it. Outperforms Node.js Express in benchmarks.

Rich NuGet Ecosystem

180,000+ packages. Entity Framework, SignalR, Dapper, AutoMapper, FluentValidation, MediatR — we use many of these by Day 7.

Ahead of You in This Bootcamp

Day 3: Middleware → Day 4: DI → Day 5: Full CRUD → Day 7: EF Core → Day 11: JWT Auth. C# is the thread.

C# vs JavaScript — Key Differences

Understanding what changes when you come from JS/Python

	JavaScript	C#
Typing	Dynamic — var x = 5; x = "hello"; ✓ Type changes at any time	Static — int x = 5; x = "hello"; ✗ Type is fixed forever at declaration
Error Detection	Runtime — bugs appear when that code runs, maybe in production	Compile time — bugs caught before you run. Green squiggle in VS Code immediately
null Safety	null is the default for objects. undefined, null, NaN — many falsy traps	Non-nullable by default. int can NEVER be null. int? required explicitly to allow null
Semicolons	Optional in many cases (ASI inserts them)	Required on every statement. No exceptions.
Classes	Prototype-based. class syntax is syntactic sugar over prototype chain	True class-based OOP. Interfaces, abstract classes, generics, sealed — all real language features
Arrays	[1, "a", true, null] — mixed types OK	int[] — all must be int. For mixed: object[], List<T>
Semicolon on method	No braces needed for arrow functions const f = x => x + 1	C# expression body: int F(int x) => x + 1; Semicolon required even here

Variables & Built-In Types

C# has value types (stored directly) and reference types (stored by pointer)

Value Types — stored in stack memory

```
// Whole numbers
int    age    = 25;          // 32-bit, ±2.1 billion (most common)
long   bigNum = 9_000_000L;  // 64-bit, _ is just a readability
separator
short  small  = 100;         // 16-bit, -32768 to 32767 (rarely used)
byte   b      = 255;         // 8-bit, 0–255, used in file/network I/O

// Decimals
double gpa    = 3.85;        // 64-bit, ~15 digits — most common decimal
float  temp   = 36.6f;       // 32-bit, 7 digits — f SUFFIX required
decimal price = 99.99m;      // 128-bit, EXACT — always use for money!

// Other value types
bool   isActive = true;       // true or false (lowercase only)
char   grade    = 'A';        // single character — SINGLE quotes
```

Reference Types — stored on heap

```
// string — most common reference type
string name  = "Maria Santos";  // DOUBLE quotes
string empty = string.Empty;    // preferred over ""
string? nullable = null;        // ? allows null

// Arrays — fixed size after creation
int[]    scores = { 90, 85, 92 };    // literal init
string[] days   = new string[7];     // 7 empty slots
int[]    range  = new int[] { 1,2,3 };

// var — compiler infers type (NOT dynamic!)
var city  = "Manila";  // inferred: string — FIXED
var count = 42;        // inferred: int    — FIXED
var pi    = 3.14;      // inferred: double — FIXED

// Nullable value types — add ? to allow null
int?  optId  = null;  // int OR null
bool? maybeB = null;  // true, false, OR null
```

Type Conversion

```
// IMPLICIT — safe, no data loss possible
int x = 42;
double y = x;          // int fits in double

// Parsing strings to typed values
int    age  = int.Parse("25");
double gpa  = double.Parse("3.85");
bool   flag = bool.Parse("true");
```

```
EXPLICIT CAST — may lose data, you take responsibility
double d = 9.99;
int i = (int)d;          // i = 9 — decimal truncated, not rounded!

Convert class
string s = Convert.ToString(42);
int    n = Convert.ToInt32("100");
```

Key Rule: Use decimal (not double) for any monetary value. double has floating-point imprecision: 0.1 + 0.2 ≠ 0.3 exactly.

Methods — The Building Blocks of Logic

A method is a named, reusable block of code that can accept inputs and return outputs

Method Anatomy & Signatures

```
// access  return-type  name    (parameters)
  public   double       Add     (int a, int b)
  private  void         LogError(string msg)
  public   string       GetLabel()             // no params
  public   static int   Square  (int n)        // static = no instance needed

// BLOCK BODY — for multi-step logic
public double GetAverage(List<double> grades) {
    if (grades.Count == 0) return 0;  // early return
    double total = 0;
    foreach (var g in grades) total += g;
    return total / grades.Count;
}

// EXPRESSION BODY — single return expression
public double GetAverage(List<double> grades) =>
    grades.Count == 0 ? 0 : grades.Average();

// VOID — no return value
public void PrintGreeting(string name) {
    Console.WriteLine($"Hello, {name}!");
    // no return needed for void
}

// OPTIONAL PARAMETERS — have default values
public string Greet(string name, string title = "Ms.") =>
    $"Hello, {title} {name}!";
// Greet("Ana")          → "Hello, Ms. Ana!"
// Greet("Ana", "Dr.")   → "Hello, Dr. Ana!"
```

Return Types & out Parameters

```
// VALUE returns
public int    GetCount()     => _items.Count;
public bool   IsEmpty()      => _items.Count == 0;
public double CalcTax(decimal amount) =>
    (double)amount * 0.12;

// NULL-SAFE returns — use nullable return type
public Student? FindById(int id) =>
    _students.FirstOrDefault(s => s.Id == id);
// caller MUST check for null:
var student = FindById(5);
if (student != null) Console.WriteLine(student.Name);

// MULTIPLE return values — use tuple
public (bool Success, string Message) TryCreate(string name) {
    if (string.IsNullOrEmpty(name))
        return (false, "Name is required");
    return (true, $"Created {name}");
}
// Usage:
var (ok, msg) = TryCreate("Ana");
Console.WriteLine(ok ? msg : "Failed: " + msg);

// OUT parameter — fill multiple values
public bool TryFindStudent(int id, out Student? student) {
    student = _students.FirstOrDefault(s => s.Id == id);
    return student != null;
}
if (TryFindStudent(1, out var s)) Console.Write(s.Name);
```


Control Flow — Decisions & Loops

How C# decides which code to run and how many times

Conditionals

```
// if / else if / else
if (age >= 18) {
    Console.WriteLine("Adult");
} else if (age >= 13) {
    Console.WriteLine("Teenager");
} else {
    Console.WriteLine("Child");
}

// IMPORTANT: C# has NO truthy/falsy
// WRONG: if (name) { }        — compile error
// RIGHT: if (!string.IsNullOrEmpty(name)) { }

// Ternary — one-line conditional
string label = age >= 18 ? "Adult" : "Minor";

// switch EXPRESSION (C# 8+) — preferred over switch statement
string level = age switch {
    >= 65          => "Senior",
    >= 18 and < 65  => "Adult",
    >= 13          => "Teenager",
    _              => "Child"    // default
};

// Pattern matching — check type + cast in one line
if (obj is string s && s.Length > 0) {
    Console.WriteLine($"Text: {s}");
}
```

Loops

```
// FOR — when you know the count or need index
for (int i = 0; i < scores.Length; i++) {
    Console.WriteLine($"[{i}] {scores[i]}");
}

// FOREACH — cleanest way to iterate any collection
int[] scores = { 90, 85, 92 };
foreach (var score in scores) {
    Console.WriteLine(score);  // no index available
}
// Works on arrays, List<T>, Dictionary, any IEnumerable

// WHILE — repeat while condition is true
int attempts = 0;
while (attempts < 3) {
    Console.Write("Enter password: ");
    attempts++;
}

// DO-WHILE — runs body at least once, then checks
do {
    Console.Write("Enter a positive number: ");
    input = Console.ReadLine();
} while (!int.TryParse(input, out n) || n <= 0);

// BREAK / CONTINUE
foreach (var s in students) {
    if (s.GPA < 2.0) continue;   // skip failing
    if (s.Name == "stop") break; // exit loop
    Console.WriteLine(s.Name);
}
```

Prefer foreach over for when you do not need the index — it is safer (no off-by-one errors) and more readable.

Classes & OOP — Full Anatomy

A class is a blueprint. An object is one instance created from that blueprint.

Complete Student Class — Every Feature Labelled

```
public class Student {                                      // ← "public" = accessible anywhere
    // ── PROPERTIES — auto-implemented (compiler generates hidden field + get/set)
    public int     Id      { get; set; }                    // readable + writable from anywhere
    public string  Name    { get; set; } = string.Empty;    // default value prevents null
    public string  Course  { get; private set; } = "";      // outsiders can READ, only this class can WRITE
    public List<double> Grades { get; set; } = new();       // new() = target-typed, infers List<double>

    // ── CONSTRUCTOR — runs when you write "new Student(...)"
    public Student(int id, string name, string course) {
        Id = id;  Name = name;  Course = course;            // assign constructor params to properties
    }

    // ── METHODS — actions this object can perform
    public double GetAverage() =>                           // expression body — single return
        Grades.Count == 0 ? 0 : Grades.Average();           // ternary: if no grades → 0, else → average

    public bool   IsHonor()    => GetAverage() >= 90;       // calls own method
    public void   AddGrade(double g) => Grades.Add(g);      // void — no return

    // ── OVERRIDE — customise what Console.WriteLine(student) prints
    public override string ToString() =>
        $"[{Id}] {Name} ({Course}) — Avg: {GetAverage():F2}"; // :F2 = 2 decimal places
}
```

`{ get; set; }`	— Auto-property — compiler generates hidden field + getter/setter code automatically
`= new()`	— Target-typed new — compiler infers type (List<double>) from the property declaration
`{ get; private set; }`	— Readable from outside, writable only inside this class. Encapsulation.
`=> expression`	— Expression body — shorthand for methods with a single return value
`override ToString()`	— Replace the default Object.ToString(). Called automatically by Console.WriteLine().

Access Modifiers, static & readonly

Controlling visibility and mutability is core C# discipline

Access Modifiers

```
// public — anyone can use it (most API members)
public string Name { get; set; }
public double GetAverage() => ...;

// private — only this class (default for fields)
private int _counter = 0;
private void IncrementCounter() => _counter++;

// private set — readable from outside, writable inside only
public string Course { get; private set; }

// protected — this class + subclasses
protected virtual void OnCreated() { ... }

// internal — same project/assembly, not outside
internal class DatabaseHelper { ... }

// C# naming conventions:
// _camelCase  → private fields
// PascalCase  → public properties, methods, classes
// camelCase   → local variables, parameters
```

static, const & readonly

```
// STATIC — belongs to the CLASS, not an instance
// No "new" needed to access static members
public static class MathHelper {
    public static double CircleArea(double r) => Math.PI * r * r;
}
double area = MathHelper.CircleArea(5.0);

// In controllers — static list simulates DB (Day 2 only)
// Day 7 replaces this with real PostgreSQL via EF Core
private static List<Book> _books = new();

// CONST — compile-time constant, can never change
public const int MaxGradeValue  = 100;
public const string ApiVersion  = "v1";

// READONLY — set ONCE (constructor or declaration)
private readonly string _connectionString;
private readonly DateTime _createdAt = DateTime.UtcNow;
// _connectionString can only be assigned in constructor
// After that, it is immutable — like a const but runtime
```

static vs const vs readonly — When to Use Each

Keyword	Set When	Mutable?	Use For
`const`	Compile-time	Never	Values known at compile time: max limits, API version
`readonly`	Runtime	Never (after ctor)	Connection strings, created timestamps, config values
`static`	App startup	Yes (unless readonly)	Shared state, utility methods, in-memory lists

Collections — List<T>, Dictionary<K,V>, HashSet<T>

Generic collections are type-safe containers used everywhere in C#

List<T> — dynamic array

```
var students = new List<Student>();

// Add items
students.Add(new Student(1,"Ana","CS"));
students.AddRange(new[]{              // batch
    new Student(2,"Ben","IT"),
    new Student(3,"Carlo","CS")
});
students.Insert(0, new Student(4,"Dina","IT"));

// Access
var first  = students[0];       // by index
int count  = students.Count;    // item count
bool found = students.Contains(first); // by ref

// Find
var ana = students
    .FirstOrDefault(s => s.Name == "Ana");
bool anyCS = students.Any(s => s.Course=="CS");

// Remove
students.Remove(ana!);          // by object ref
students.RemoveAt(0);           // by index
students.RemoveAll(s =>         // by condition
    s.Course == "IT");

// Sort
students.Sort((a,b) =>          // custom comparer
    a.Name.CompareTo(b.Name));
```

Dictionary<K,V> — key lookup

```
// K=key type, V=value type
var scores = new Dictionary<int, Student>();

// Add
scores[1] = new Student(1,"Ana","CS");
scores.Add(2, new Student(2,"Ben","IT")); // throws if key
exists

// SAFE read — always use TryGetValue
if (scores.TryGetValue(1, out var student)) {
    Console.WriteLine(student.Name); // "Ana"
}

// Direct read (throws KeyNotFoundException!)
var s = scores[99]; // DANGER if 99 not present

// Check before access
if (scores.ContainsKey(3)) { ... }

// Update
scores[1] = new Student(1,"Ana Updated","CS");

// Remove
scores.Remove(2);

// Iterate — gives KeyValuePair<K,V>
foreach (var kvp in scores) {
    Console.WriteLine(
        $"Key:{kvp.Key} → {kvp.Value.Name}");
}

// Just keys or values
foreach (var key in scores.Keys) { ... }
foreach (var val in scores.Values) { ... }
```

HashSet<T> — unique items

```
// Automatically prevents duplicates
var tags = new HashSet<string>();

tags.Add("csharp");
tags.Add("dotnet");
tags.Add("csharp");   // silently ignored!
Console.WriteLine(tags.Count); // 2

// Membership check — O(1) very fast
tags.Contains("dotnet"); // true

// Set operations (like math set theory)
var a = new HashSet<int> {1,2,3,4,5};
var b = new HashSet<int> {3,4,5,6,7};

// Union — all items from both
a.UnionWith(b);      // {1,2,3,4,5,6,7}

// Intersection — items in BOTH
a.IntersectWith(b);  // {3,4,5}

// Difference — in a but NOT in b
a.ExceptWith(b);     // {1,2}

// Is subset / superset
a.IsSubsetOf(b);
a.IsSupersetOf(b);

// Convert to sorted List
var sorted = tags.OrderBy(t => t).ToList();
```

Choose based on need: List = ordered sequence | Dictionary = lookup by key | HashSet = uniqueness + fast contains

LINQ — Language Integrated Query

"SQL for your C# collections." Same WHERE/SELECT/ORDER BY concepts, C# syntax.

Filtering, Sorting & Chaining

```
var students = new List<Student> {
    new(1,"Ana","CS",3.9), new(2,"Ben","IT",3.1),
    new(3,"Carlo","CS",3.6), new(4,"Diana","IT",3.8)
};

// WHERE — keep items matching condition
var csStudents = students
    .Where(s => s.Course == "CS")
    .ToList();
// → [Ana(3.9), Carlo(3.6)]

// ORDERBY / ORDERBYDESCENDING
var topFirst = students
    .OrderByDescending(s => s.GPA)
    .ToList();
// → Ana, Diana, Carlo, Ben

// TAKE + SKIP — for pagination (Day 5)
var page1 = students
    .Skip(0).Take(2).ToList();  // items 0–1
var page2 = students
    .Skip(2).Take(2).ToList();  // items 2–3

// CHAIN multiple operations — SQL style
var topCS = students
    .Where(s => s.Course == "CS")
    .OrderByDescending(s => s.GPA)
    .Take(1)
    .ToList();  // → [Ana(3.9)]
```

Projection, Aggregates & Terminal Ops

```
// SELECT — project/transform each item
var names = students
    .Select(s => s.Name)
    .ToList();
// → ["Ana","Ben","Carlo","Diana"]

// Anonymous object projection (common for API DTOs)
var dtos = students.Select(s => new {
    s.Id, s.Name,
    IsHonor = s.GPA >= 3.7
}).ToList();

// AGGREGATE FUNCTIONS
int    count = students.Count();                  // 4
double avg   = students.Average(s => s.GPA);      // 3.6
double max   = students.Max(s => s.GPA);          // 3.9
double min   = students.Min(s => s.GPA);          // 3.1

// ELEMENT OPERATIONS
var first = students.First();          // throws if empty
var safe  = students.FirstOrDefault(); // null if empty — USE THIS
var byId  = students.FirstOrDefault(s => s.Id == 1);  // null safe

// BOOLEAN CHECKS
bool anyHonor = students.Any(s => s.GPA >= 3.7);  // true
bool allPass  = students.All(s => s.GPA >= 2.0);  // true

// .ToList() ALWAYS needed at the end of LINQ chains
// Without it, query is LAZY — runs each time you iterate
```

Day 7 (EF Core): the EXACT same LINQ syntax runs against PostgreSQL — translated to SQL automatically. Learn it once, use it everywhere.

Lambda Expressions — The => Syntax

A lambda is an anonymous function written inline. Used everywhere in LINQ.

Lambda Anatomy

```
// General form:
//   (parameters) => expression  OR  { block }

// One parameter — parentheses optional
s => s.GPA >= 3.7           // "student s: return GPA >= 3.7"
s => s.Name.ToUpper()       // "student s: return Name uppercased"

// Two parameters
(a, b) => a + b             // sum
(a, b) => a.CompareTo(b)    // comparison

// No parameters
() => DateTime.UtcNow       // returns current time

// Block body — multiple statements
s => {
    var avg = s.GPA;
    return avg >= 3.7 && s.Course == "CS";
};

// Lambdas in LINQ:
students.Where(s => s.GPA >= 3.7)
//             ^---- lambda passed as argument
//       "for each student s, keep it if GPA >= 3.7"
```

Func<> and Action<> Delegates

```
// Func<TIn, TOut> — a function that takes TIn and returns TOut
Func<Student, double> getGpa = s => s.GPA;
Func<int, int, int>   add    = (a, b) => a + b;
Func<string, bool>    isLong = s => s.Length > 10;

// Usage
double gpa = getGpa(students[0]);  // 3.9
int    sum = add(3, 7);            // 10

// Action<T> — takes T, returns nothing (void)
Action<string> log   = msg => Console.WriteLine(msg);
Action<int>    print = n   => Console.WriteLine(n);
log("Hello");  // prints Hello

// What LINQ methods actually accept:
// .Where(Func<T, bool> predicate)
// .Select(Func<T, TResult> selector)
// .OrderBy(Func<T, TKey> keySelector)

// You can store and reuse lambdas:
Func<Student,bool> honorFilter = s => s.GPA >= 3.7;
var honors = students.Where(honorFilter).ToList();
```

Reading LINQ with Lambdas — How to Say It Out Loud

```
students                                    "start with the students collection"
    .Where(s => s.Course == "CS")           "keep only students where course is CS"
    .OrderByDescending(s => s.GPA)          "sort those by GPA, highest first"
    .Select(s => new { s.Name, s.GPA })     "project to just Name and GPA"
    .Take(3)                                 "take the top 3"
    .ToList();                               "execute and return as a List"   ← REQUIRED!
```

ACTIVITY — 20 MIN

Console App — Student Grade Tracker

PRE-CHECK: Open terminal → type: dotnet --version → should show 8.x.x
If not installed: dotnet.microsoft.com/download → .NET 8 SDK → install → reopen terminal

STEPS:

1. Terminal → dotnet new console -n GradeTracker → cd GradeTracker → code .
2. In Program.cs: delete all content and create a Student class with:
   - Id (int), Name (string), Course (string), Grades (List<double>)
   - Constructor: public Student(int id, string name, string course)
   - GetAverage() → returns Grades.Average() or 0 if empty
   - IsHonor() → returns GetAverage() >= 87.0
   - Override ToString() → e.g. "[1] Ana (CS) — Avg: 91.83"
3. Create at least 4 students with different course values and grade sets
4. Use LINQ to produce these exact outputs:
   a) Highest average student (any course)
   b) All honor students (GPA >= 87), sorted descending
   c) Count of students per course (group by Course)
   d) Full ranking — all students sorted highest to lowest GPA
5. Run with: dotnet run

Expected output format:

Top: Ana Reyes (CS) — Avg: 91.83
Honors: Ana Reyes, Diana Cruz
CS: 2 students | IT: 2 students
Rank 1: Ana Reyes 91.83 | Rank 2: Diana Cruz 89.50 | ...

ASP.NET Core Web API — Project Setup

Foundation for Days 3–5

Everything you need to run your first API in under 2 minutes

Terminal Commands

```
# 1. Create the project
dotnet new webapi -n LibraryApi

# 2. Enter the folder
cd LibraryApi

# 3. Open in VS Code
code .

# 4. Run the server
dotnet run

# Look for this in the terminal output:
# Now listening on: https://localhost:5001
# Copy that port — you need it for Postman

# 5. Hot-reload (auto-restart on save)
dotnet watch

# 6. Add packages later (Day 7)
dotnet add package Npgsql.EntityFrameworkCore.PostgreSQL
```

Project Files Explained

Program.cs	App startup, middleware pipeline, DI container
Controllers/	Your API endpoint classes go here
appsettings.json	DB connection strings, logging, custom config
appsettings.Development.json	Dev-only overrides for appsettings.json
LibraryApi.csproj	Project file — NuGet packages, .NET version
Properties/launchSettings.json	Dev server ports + environment variables

Program.cs — Every Line Explained (you will extend this in Day 3)

```
var builder = WebApplication.CreateBuilder(args);  // setup config, logging, DI container
builder.Services.AddControllers();                 // register [ApiController] classes (Day 4: you add more services here)
builder.Services.AddEndpointsApiExplorer();        // make endpoints discoverable (for documentation tools)
var app = builder.Build();                         // finalize — after this line, no more services
app.MapControllers();                              // wire HTTP router → controller action methods
```

Day 3 adds middleware here (app.Use...). Day 4 adds services (builder.Services.AddScoped<>). You will revisit Program.cs every day.

Models & Project Structure

📍 Clean layering — Day 3 deep dive

Organise your code from Day 2 so it scales to Day 12

Folder Structure Convention

```
LibraryApi/
├── Controllers/          ← HTTP routing + request handling
│   └── BooksController.cs
├── Models/               ← Entity/data classes (what DB holds)
│   ├── Book.cs
│   └── Author.cs
├── DTOs/                 ← API shapes (what clients send/receive)
│   ├── BookCreateDto.cs
│   └── BookResponseDto.cs
├── Services/             ← Business logic (Day 4+)
│   └── IBookService.cs
├── Program.cs            ← Startup
└── appsettings.json

# Day 2: Models + Controllers (simple)
# Day 3: Add middleware + layering
# Day 4: Add Services/ + DI
# Day 5: Full CRUD with validation
# Day 7: Add Data/ for EF Core
```

Models/Book.cs

```
namespace LibraryApi.Models;

public class Book {
    public int    Id        { get; set; }
    public string Title     { get; set; } = string.Empty;
    public string Author    { get; set; } = string.Empty;
    public string Genre     { get; set; } = string.Empty;
    public int    Year      { get; set; }
    public bool   Available { get; set; } = true;
}
```

What is a DTO and Why?

```
// DTO = Data Transfer Object
// Shape the data for a specific API operation
// POST body — what CLIENT sends (no Id, they don't know it yet)
public class BookCreateDto {
    public string Title  { get; set; } = "";
    public string Author { get; set; } = "";
    public string Genre  { get; set; } = "";
}

// GET response — what SERVER returns (includes Id)
public class BookResponseDto {
    public int    Id     { get; set; }
    public string Title  { get; set; } = "";
    public bool Available { get; set; }
    // Internal fields like audit timestamps NOT exposed
}
```

Day 2: use Book directly to keep it simple. Day 3 onwards: separate DTOs from Models. Start the folder structure now so refactoring is easy.

Controllers & Attribute Routing

How incoming HTTP requests find the right C# method

BooksController.cs — GET Endpoints

```
[ApiController]              // ← enables auto-validation + attribute routing
[Route("api/[controller]")]  // ← [controller] = "books" (class name - "Controller")
public class BooksController : ControllerBase {

    // Static list — simulates DB for now (Day 7: replaced by EF Core + PostgreSQL)
    private static List<Book> _books = new() {
        new Book { Id=1, Title="Clean Code",   Author="Robert Martin", Genre="Programming",  Available=true  },
        new Book { Id=2, Title="C# in Depth",  Author="Jon Skeet",     Genre="Programming",  Available=false },
        new Book { Id=3, Title="Dune",         Author="Frank Herbert", Genre="Fiction",      Available=true  },
    };

    // GET /api/books
    [HttpGet]
    public IActionResult GetAll() => Ok(_books);

    // GET /api/books/1        ← {id} is a ROUTE PARAMETER
    [HttpGet("{id:int}")]    // :int constrains type — won't match /api/books/abc
    public IActionResult GetById(int id) {
        var book = _books.FirstOrDefault(b => b.Id == id);
        return book == null ? NotFound() : Ok(book);
    }

    // GET /api/books/search?genre=Fiction&available=true  ← QUERY PARAMETERS
    [HttpGet("search")]
    public IActionResult Search(
        [FromQuery] string? genre,       // ?genre=Fiction  (optional)
        [FromQuery] bool?   available) { // ?available=true (optional)
        var result = _books.AsEnumerable();
        if (!string.IsNullOrEmpty(genre)) result = result.Where(b => b.Genre == genre);
        if (available.HasValue) result = result.Where(b => b.Available == available.Value);
        return Ok(result.ToList());
    }
}
```

Attribute Reference

`[ApiController]`
Auto model validation
+ attribute routing required

`[Route("api/[controller]")]`
[controller] → class name
minus "Controller" suffix

`[HttpGet]`

Maps to GET /api/books

`[HttpGet("{id}")]`

Route param: GET /api/books/5

`[HttpGet("{id:int}")]`
Type-constrained route param
rejects non-integer paths

`[HttpGet("search")]`
Named sub-route:
GET /api/books/search

`[FromQuery] string? genre`
Binds ?genre= from URL
? means optional

`ControllerBase`
Provides: Ok(), NotFound()
Created(), BadRequest(), NoContent()

Full CRUD — POST, PUT, PATCH, DELETE

Write operations with correct status codes and [FromBody] deserialization

POST, PUT, PATCH, DELETE — Complete Implementation

```
// POST /api/books  — CREATE  → 201 Created  (body is the JSON you send)
[HttpPost]
public IActionResult Create([FromBody] Book book) {    // [FromBody] reads + deserializes JSON body
    book.Id = _books.Count == 0 ? 1 : _books.Max(b => b.Id) + 1;
    _books.Add(book);
    return CreatedAtAction(nameof(GetById),            // 201 + Location: /api/books/4
        new { id = book.Id }, book);
}

// PUT /api/books/1  — FULL REPLACE  → 200 OK  (send ALL fields, missing ones reset to default)
[HttpPut("{id:int}")]
public IActionResult FullUpdate(int id, [FromBody] Book updated) {
    var existing = _books.FirstOrDefault(b => b.Id == id);
    if (existing == null) return NotFound();           // 404 if not found
    existing.Title = updated.Title;  existing.Author = updated.Author;
    existing.Genre = updated.Genre;  existing.Year   = updated.Year;
    existing.Available = updated.Available;
    return Ok(existing);                               // 200 with updated book
}

// PATCH /api/books/1  — PARTIAL UPDATE  → 200 OK  (only send fields you want to change)
[HttpPatch("{id:int}")]
public IActionResult PartialUpdate(int id, [FromBody] BookPatchDto patch) {
    var existing = _books.FirstOrDefault(b => b.Id == id);
    if (existing == null) return NotFound();
    if (patch.Title     != null) existing.Title     = patch.Title;    // null = "don't change"
    if (patch.Available != null) existing.Available = patch.Available.Value;
    return Ok(existing);
}

// DELETE /api/books/1  — REMOVE  → 204 No Content  (no response body)
[HttpDelete("{id:int}")]
public IActionResult Delete(int id) {
    var book = _books.FirstOrDefault(b => b.Id == id);
    if (book == null) return NotFound();               // 404 if already gone
    _books.Remove(book);
    return NoContent();                                // 204 — nothing to return
}
```

PUT = replace the whole thing. PATCH = change only what you send. Day 5 builds on this with full validation and error handling middleware.

IActionResult & HTTP Status Codes

Return the right code — clients depend on it for error handling

ControllerBase Helper Methods

```
// ── 2xx SUCCESS ──────────────────────────────
return Ok(data);                  // 200 — GET/PUT/PATCH succeed
return Created(uri, data);        // 201 — with explicit URI
return CreatedAtAction(           // 201 — URI built from action name
(preferred)
    nameof(GetById),
    new { id = obj.Id }, obj);
return NoContent();               // 204 — DELETE success, no body
return Accepted();                // 202 — async operation started

// ── 4xx CLIENT ERRORS ────────────────────────
return NotFound();                // 404 — resource doesn't exist
return NotFound("Book not found"); // 404 + custom message
return BadRequest();              // 400 — malformed request
return BadRequest(ModelState);  // 400 + validation error details
return Unauthorized();            // 401 — not authenticated (Day 11)
return Forbid();                  // 403 — authenticated, no permission
return Conflict();                // 409 — duplicate / state conflict
return UnprocessableEntity(err); // 422 — semantically wrong body

// ── CUSTOM STATUS ────────────────────────────
return StatusCode(418, "I'm a teapot");

// ── TYPED RESULT (more explicit) ─────────────
return TypedResults.Ok(book);
return TypedResults.NotFound();
```

Quick Reference

200 OK	— GET, PUT, PATCH success
201 Created	— POST success — new resource
204 No Content	— DELETE success — no body
400 Bad Request	— Invalid body / failed validation
401 Unauthorized	— Missing or invalid auth (Day 11)
403 Forbidden	— Auth ok but no permission
404 Not Found	— Resource does not exist
409 Conflict	— Duplicate / violates state
422 Unprocessable	— Semantically wrong body
500 Internal Error	— Unhandled server exception

Always return 404 for missing resources — never return 200 with null or an empty body. Clients cannot distinguish success from failure.

Controllers & Attribute Routing

How incoming HTTP requests find the right C# method

BooksController.cs with full CRUD

```
using Microsoft.AspNetCore.Mvc;
using LibraryApi.Models;

namespace LibraryApi.Controllers;

[ApiController]
[Route("api/[controller]")]
public class BooksController : ControllerBase
{
    private static List<Book> _books = new()
    {
        new Book { Id = 1, Title = "Clean Code",          Author = "Robert Martin", Genre = "Programming", Year = 2008, Available = true  },
        new Book { Id = 2, Title = "C# in Depth",         Author = "Jon Skeet",     Genre = "Programming", Year = 2019, Available = false },
        new Book { Id = 3, Title = "Dune",                Author = "Frank Herbert", Genre = "Fiction",     Year = 1965, Available = true  },
        new Book { Id = 4, Title = "The Pragmatic Programmer", Author = "David Thomas", Genre = "Programming", Year = 1999, Available = false },
        new Book { Id = 5, Title = "Sapiens",             Author = "Yuval Harari",  Genre = "History",     Year = 2011, Available = true  },
    };

    // GET /api/books
    [HttpGet]
    public IActionResult GetAll() => Ok(_books);

    // GET /api/books/5
    [HttpGet("{id:int}")]
    public IActionResult GetById(int id)
    {
        var book = _books.FirstOrDefault(b => b.Id == id);
        return book == null ? NotFound() : Ok(book);
    }

    // GET /api/books/search?genre=fiction&available=true
    [HttpGet("search")]
    public IActionResult Search(
        [FromQuery] string? genre,
        [FromQuery] bool? available,
        [FromQuery] string? author)
    {
        var result = _books.AsQueryable();
        if (!string.IsNullOrEmpty(genre))
            result = result.Where(b => b.Genre.Equals(genre, StringComparison.OrdinalIgnoreCase));
        if (available.HasValue)
            result = result.Where(b => b.Available == available.Value);
        if (!string.IsNullOrEmpty(author))
            result = result.Where(b => b.Author.Contains(author, StringComparison.OrdinalIgnoreCase));
        return Ok(result.ToList());
    }

    // GET /api/books/stats
    [HttpGet("stats")]
    public IActionResult GetStats()
    {
        var stats = new
        {
            Total     = _books.Count,
            Available = _books.Count(b => b.Available),
            Borrowed  = _books.Count(b => !b.Available),
            ByGenre   = _books
                .GroupBy(b => b.Genre)
                .Select(g => new { Genre = g.Key, Count = g.Count() })
                .OrderByDescending(g => g.Count)
                .ToList()
        };
        return Ok(stats);
    }

    // POST /api/books
    [HttpPost]
    public IActionResult Create([FromBody] Book book)
    {
        book.Id = _books.Count == 0 ? 1 : _books.Max(b => b.Id) + 1;
        _books.Add(book);
        return CreatedAtAction(nameof(GetById), new { id = book.Id }, book);
    }

    // PUT /api/books/1
    [HttpPut("{id:int}")]
    public IActionResult FullUpdate(int id, [FromBody] Book updated)
    {
        var existing = _books.FirstOrDefault(b => b.Id == id);
        if (existing == null) return NotFound();
        existing.Title     = updated.Title;
        existing.Author    = updated.Author;
        existing.Genre     = updated.Genre;
        existing.Year      = updated.Year;
        existing.Available = updated.Available;
        return Ok(existing);
    }

    // PATCH /api/books/1
    [HttpPatch("{id:int}")]
    public IActionResult PartialUpdate(int id, [FromBody] BookPatchDto patch)
    {
        var existing = _books.FirstOrDefault(b => b.Id == id);
        if (existing == null) return NotFound();
        if (patch.Title     != null) existing.Title     = patch.Title;
        if (patch.Available != null) existing.Available = patch.Available.Value;
        return Ok(existing);
    }

    // DELETE /api/books/1
    [HttpDelete("{id:int}")]
    public IActionResult Delete(int id)
    {
        var book = _books.FirstOrDefault(b => b.Id == id);
        if (book == null) return NotFound();
        _books.Remove(book);
        return NoContent();
    }
}
```

Attribute Reference

`[ApiController]`
Auto model validation
+ attribute routing required

`[Route("api/[controller]")]`
[controller] → class name
minus "Controller" suffix

`[HttpGet]`

Maps to GET /api/books

`[HttpGet("{id}")]`

Route param: GET /api/books/5

`[HttpGet("{id:int}")]`
Type-constrained route param
rejects non-integer paths

`[HttpGet("search")]`
Named sub-route:
GET /api/books/search

`[FromQuery] string? genre`
Binds ?genre= from URL
? means optional

`ControllerBase`
Provides: Ok(), NotFound()
Created(), BadRequest(), NoContent()

Data Annotations — Automatic Request Validation

[ApiController] auto-returns 400 Bad Request when annotations fail — zero extra code

Book.cs with Full Validation

```
using System.ComponentModel.DataAnnotations;

namespace LibraryApi.Models;

public class Book
{
    public int Id { get; set; }

    [Required(ErrorMessage = "Title is required")]
    [StringLength(255, MinimumLength = 1, ErrorMessage = "Title must be 1-255 characters")]
    public string Title { get; set; } = string.Empty;

    [Required(ErrorMessage = "Author is required")]
    [StringLength(150, MinimumLength = 2)]
    public string Author { get; set; } = string.Empty;

    [StringLength(80)]
    public string Genre { get; set; } = string.Empty;

    [Range(1000, 2030, ErrorMessage = "Year must be between 1000 and 2030")]
    public int Year { get; set; }

    public bool Available { get; set; } = true;

    [Url(ErrorMessage = "CoverImageUrl must be a valid URL")]
    public string? CoverImageUrl { get; set; }
}

public class BookPatchDto
{
    public string? Title     { get; set; }
    public bool?   Available { get; set; }
}
```

Annotation Reference

`[Required]`
Must be present and non-empty

`[StringLength(max,Min=n)]`
Length between Min and max

`[Range(min,max)]`
Number between min and max

`[MinLength(n)]`
Array/string min elements

`[MaxLength(n)]`
Array/string max elements

`[EmailAddress]`
Valid email format

`[Url]`
Valid URL with http(s)://

`[RegularExpression]`
Must match regex pattern

`[Compare("Other")]`
Must equal another property

[ApiController] makes validation automatic. Send an empty title → ASP.NET returns 400 before your method code even runs. Day 5 adds FluentValidation for complex rules.

Testing Your API with Postman

📍 Used every day from Day 2

Postman is the industry-standard API testing tool — used throughout this bootcamp

1	**Install** Download from postman.com/downloads → install → sign up free → open the app
2	**New Request** Click "+" tab → choose method (GET) → paste URL: https://localhost:5001/api/books
3	**GET Test** Click Send → should see 200 OK with JSON array in the response panel below
4	**POST Test** Change method to POST → Body tab → raw → JSON → paste request body → Send → expect 201 Created
5	**PUT Test** Method PUT → URL: /api/books/1 → Body: full JSON object with all fields → Send → expect 200
6	**DELETE Test** Method DELETE → URL: /api/books/1 → (no body) → Send → expect 204 No Content
7	**SSL Issue Fix** If "SSL certificate error": Settings → General → turn OFF SSL certificate verification

Postman — Request Bodies & Reading Responses

How to send JSON, read status codes, and inspect headers

POST /api/books — Create body

```
// Method: POST
// URL: https://localhost:5001/api/books
// Headers: Content-Type: application/json  (Postman sets this when you pick raw+JSON)
// Body → raw → JSON:
{
  "title": "The Pragmatic Programmer",
  "author": "David Thomas",
  "genre": "Programming",
  "year": 1999,
  "available": true
}
// Expected response: 201 Created
// Check the "Location" header in Response Headers tab
```

PATCH /api/books/1 — Partial Update

```
// Method: PATCH
// URL: https://localhost:5001/api/books/1
// Body → raw → JSON:
{
  "available": false
}
// Only "available" changes — title, author etc. stay the same
// Expected response: 200 OK with updated book

// Validation failure test:
// POST with empty title:
{ "title": "", "author": "", "year": 9999 }
// Expected: 400 Bad Request + validation errors
```

Reading the Postman Response Panel

Status code	`200 OK / 201 Created / 404 Not Found`	Top right of response panel — always check this first
Body tab	`The JSON response body`	Main output — what the server returned
Headers tab	`Response headers including Location`	Check Location header after 201 Created

Always check the status code FIRST in Postman. A 400 or 404 is not a bug in Postman — it is your API telling you something is wrong.

ACTIVITY — 20 MIN

Build Your First Web API — StudentApi + Postman Tests

Create project: dotnet new webapi -n StudentApi → cd StudentApi → code .

1. Create Models/Student.cs: Id (int), Name (string), Course (string), GPA (double)
   Add: [Required] on Name and Course, [Range(0.0, 4.0)] on GPA

2. Create Controllers/StudentsController.cs with [ApiController] and [Route("api/[controller]")]:
   - GET /api/students → return all (_students list)
   - GET /api/students/{id:int} → find by id, 404 if not found
   - POST /api/students → [FromBody], generate Id, return 201 CreatedAtAction
   - GET /api/students/search → [FromQuery] string? course, bool? honorOnly (GPA >= 3.7)

3. Pre-populate the static list with 4 students across 2 courses

4. Run: dotnet run (note the localhost port)

5. Open Postman → test all 4 endpoints:
   - GET all → GET id=1 → GET id=99 (expect 404)
   - POST valid student → check 201 + Location header
   - POST invalid (empty name) → check 400 + error message
   - GET search?course=CS → GET search?honorOnly=true

Bonus: Add PUT and DELETE and test full CRUD in Postman

🛠 2-HOUR HANDS-ON LAB

Book Inventory Web API — Console App to Full CRUD + Postman

1

Part 1 — Console: Student Grades (30 min)

Build the GradeTracker console app from Activity 1. Add GroupBy course, top 3 per course, and export to a formatted string. dotnet run must show clean output.

2

Part 2 — API: Models & GET Endpoints (30 min)

Create BookInventoryApi. Add Book model with annotations. BooksController: GET all, GET by id, GET search (genre + available filters). Test all 3 in Postman.

3

Part 3 — API: Write Operations (40 min)

Add POST (201 + Location header), PUT (full update), PATCH (available only), DELETE (204). Test all in Postman. Test 400 validation with invalid POST body.

4

Part 4 — Polish & Collections (20 min)

Add GET /api/books/stats returning count by genre using LINQ GroupBy. Add in-memory Author list. Test Postman Collection with all saved requests end-to-end.

See the Day 2 Reviewer & Solution Guide for complete step-by-step code and expected output.

References & Further Reading

Day 2

[1] **C# Language Reference**
Microsoft Docs · learn.microsoft.com/dotnet/csharp/language-reference

[2] **ASP.NET Core Web API Tutorial**
Microsoft Docs · learn.microsoft.com/aspnet/core/tutorials/first-web-api

[3] **C# in Depth, 4th Edition**
Jon Skeet — Manning · manning.com/books/c-sharp-in-depth-fourth-edition

[4] **LINQ Documentation**
Microsoft Docs · learn.microsoft.com/dotnet/csharp/linq

[5] **Data Annotations**
ComponentModel.DataAnnot. · learn.microsoft.com/dotnet/api/system.componentmodel.dataannotations

[6] **ASP.NET Core Attribute Routing**
Microsoft Docs — Routing · learn.microsoft.com/aspnet/core/mvc/controllers/routing

[7] **Postman Documentation**
postman.com/docs · learning.postman.com/docs/getting-started

[8] **async / await in C#**
Microsoft Docs · learn.microsoft.com/dotnet/csharp/asynchronous-programming

[9] **.NET 8 SDK Download**
dotnet.microsoft.com · dotnet.microsoft.com/download